

Optimal $(D - J)$ -monotonic priority assignment

Areej Zuhily*, Alan Burns

University of York, Department of Computer Science, York YO10 5DD, UK

Received 21 November 2006; received in revised form 18 April 2007

Available online 29 April 2007

Communicated by A.A. Bertossi

Keywords: Real-time systems; Fixed priority scheduling; Priority assignment; Optimality

1. Introduction

Each concurrent task in a real time system gives rise to a series of jobs that are executing in a repeating sequence. All tasks are characterised by their (relative) deadlines and their periods. For most systems, the order of execution of jobs is controlled by their priorities that are assigned using a specified scheduling scheme. In the popular fixed priority scheduling scheme, all jobs from each task are assigned the same priority [1]. This means that the priority of each task is fixed relative to the other tasks in the system. Liu and Layland [2] introduced, assuming deadline and period are identical for each task in the system, an optimal priority assignment that is called rate monotonic (RM); where the shorter the period of the task, the higher priority it is assigned. Later, Leung and Whitehead [3] allowed a task's deadline to be less than the period for each task in the system and introduced another optimal priority assignment that is called deadline monotonic (DM); where shorter deadline the task has, the higher priority it is assigned.

The period of the task represents the minimum time between two successive releases of that task. When the

shortest time between successive releases of a task is shorter than the normal repeating time, we say that the task is subject to *release jitter*. The Release Jitter, RJ, of a task, by definition, is the maximum deviation of successive task releases from its period. Tindell in his thesis [4] addressed the problem of release jitter. The following scenario illustrates the phenomena:

Suppose there are, on different processors, two related tasks: a periodic one τ_j and a sporadic one τ_i with the same minimum inter arrival time (period). Task τ_j calls τ_i as soon as τ_j has finished its execution. Due to system load, τ_j does not finish its first execution until the end of its period; while it executes at the very beginning of its next period. As a result, τ_i is released twice within its period instead of once (i.e., the time between the two successive releases is less than the usual minimum inter arrival time of the sporadic task τ_i).

It is obvious that as a result of this scenario, the amount of interference from task τ_i , on lower priority tasks on the same processor, may be greater than that assumed for a purely periodic task.

It has been observed that neither DM nor RM priority assignments are optimal in the case of jitter [4]. The alternative to a simple priority assignment rule is

* Corresponding author.

E-mail addresses: ehar@cs.york.ac.uk (A. Zuhily),
burns@cs.york.ac.uk (A. Burns).

to employ the general optimal assignment technique of Audsley [5,6]. This technique is summarised as follows. For a task set $\phi = \{\tau_1, \tau_2, \dots, \tau_n\}$, first attempt to find a task τ_j that is feasible at priority level n (i.e., the lowest priority). Next, remove τ_j from ϕ and find another feasible task at priority $n - 1$. Successively, feasible tasks will be found at priorities n to 1. If a feasible task, at any priority level i , could not be found, no feasible priority assignment function exists. The full details of this approach can be found in the above references.

Although this general priority assignment technique can be applied to tasks with jitter, Burns et al. [7] mentioned that priorities should be assigned according to $(D - J)$ -monotonic order as DM is no longer optimal when tasks have jitter. No proof of the optimality of $(D - J)$ -monotonic has been provided until now. In this paper, we prove the optimality of $(D - J)$ -monotonic priority assignment. This proof is given when the deadline is less than or equal to the period for each task in the system.

2. Notation

The following notation is used in this paper:

- n Number of tasks in the system.
- τ_i The i th task in the system.
- R_i Worst-case response time of the i th task.
- C_i Worst-case execution time of the i th task.
- D_i Deadline of the i th task.
- T_i Period of the i th task.
- J_i Release jitter of the i th task.
- P_i Priority of the i th task.
- $(D - J)$ -monotonic Priority assignment where the task that has lower value of $(D - J)$ is assigned a higher priority. In other words, $\forall i, j \in 1..n$:

$$D_i - J_i < D_j - J_j \Rightarrow P_i > P_j.$$

3. Feasibility analysis

In this section, we introduce the exact worst case response time formula, that was given by Joseph and Pandya [8], to test the schedulability of a system. Audsley and his colleagues [9], in 1993, improved this formula and gave a simple way for solving it using a recurrence relationship. The response time equation takes the following form:

$$R_i = C_i + \sum_{j \in hp(i)} \left\lceil \frac{R_j}{T_j} \right\rceil C_j.$$

This equation has the following meaning: the worst case response time (R_i) of a task τ_i is the execution time of that task added to the interferences from all higher priority tasks, where $hp(i)$ represents the set of all tasks that have higher priority than task τ_i .

However, when tasks suffer from release jitter, the response time equation is modified to include a release jitter term [7]:

$$R_i = C_i + \sum_{j \in hp(i)} \left\lceil \frac{R_j + J_j}{T_j} \right\rceil C_j. \quad (1)$$

4. Optimality of $(D - J)$ -monotonic

In this section, we prove the optimality of $(D - J)$ -monotonic priority assignment. Let Q be any system of n independent tasks that are identified by their period T_i , deadline D_i , and release jitter J_i . In addition, assume that W is an arbitrary priority assignment scheme that can be applied to Q .

Theorem. *The priority assignment scheme $(D - J)$ -monotonic is an optimal priority scheme in the sense that if any task set, Q , is schedulable by priority scheme, W , it is also schedulable by $(D - J)$ -monotonic priority ordering.*

Proof. To prove the optimality of $(D - J)$ -monotonic priority assignment, the priorities of Q (as assigned by W) will be transformed until the ordering is $(D - J)$ -monotonic while preserving schedulability. Let τ_i and τ_j be two tasks with successive priorities in Q such that under W : $P_i > P_j$ and $D_i - J_i > D_j - J_j$. If it is not possible to find tasks τ_i and τ_j with this property then the tasks are already in $(D - J)$ order. Define scheme W' to be identical to W except that tasks τ_i and τ_j are swapped. The schedulability of all tasks that have higher priorities than τ_i or lower priorities than τ_j are not affected by swapping the two tasks τ_i and τ_j . Moreover, the schedulability of task τ_j will also not be affected by the swap since it will have higher priority than before and therefore it will suffer less interference. In remain to prove that task τ_i is still schedulable under W' .

Let R_j^W be the response time of task τ_j under scheme W , and $R_j^{W'}$ be the response time of task τ_j under scheme W' . It can be seen that $R_j^W \leq D_j - J_j$ because τ_j is schedulable and in the worst case may not be released until time $t = J_j$. In addition, it is given that $D_j - J_j < D_i - J_i \leq D_i \leq T_i$. Therefore, task τ_i only interferes once during the execution of τ_j (under W).

So, the worst case response time of task τ_j can be split, under scheme W into

$$R_j^W = C_j + C_i + \sum_{k \in S} \left\lceil \frac{R_j^W + J_k}{T_k} \right\rceil C_k, \quad (2)$$

where S is the set of tasks that have higher priority than τ_i under W (which is equal to the set of higher priority than τ_j under W'). Eq. (2) can be rewritten as

$$R_j^W - C_j = C_i + \sum_{k \in S} \left\lceil \frac{R_j^W + J_k}{T_k} \right\rceil C_k. \quad (3)$$

The response time equation of the task τ_i under scheme W' is given by

$$R_i^{W'} = C_i + \sum_{k \in hp(i)} \left\lceil \frac{R_i^{W'} + J_k}{T_k} \right\rceil C_k.$$

Hence,

$$R_i^{W'} = C_i + \left\lceil \frac{R_i^{W'} + J_j}{T_j} \right\rceil C_j + \sum_{k \in S} \left\lceil \frac{R_i^{W'} + J_k}{T_k} \right\rceil C_k. \quad (4)$$

Assuming Lemma 1 (given below), R_j^W is a solution of this equation for $R_i^{W'}$; which means that $R_i^{W'} \leq R_j^W$.

On the other hand, we have that $R_j^W \leq D_j - J_j$ as well as $D_j - J_j < D_i - J_i$.

Therefore,

$$R_i^{W'} < (D_i - J_i)$$

which implies that task τ_i is schedulable after swapping tasks τ_i and τ_j .

Now, priority scheme W' can be transformed to W'' by choosing two more tasks that are in the wrong order for $(D - J)$ -monotonic, and swapping them. Each such swap preserves schedulability. Eventually, there will be no more tasks to swap and the priority ordering will be exactly $(D - J)$ -monotonic. Hence, $(D - J)$ -monotonic is optimal. $\square$

Lemma 1. *The response time of the task τ_j under scheme W , R_j^W , is a solution of Eq. (4).*

Proof. R_j^W is substituted into the right side of Eq. (4):

$$\begin{aligned} C_i + \left\lceil \frac{R_j^W + J_j}{T_j} \right\rceil C_j + \sum_{k \in S} \left\lceil \frac{R_j^W + J_k}{T_k} \right\rceil C_k \\ = R_j^W - C_j + \left\lceil \frac{R_j^W + J_j}{T_j} \right\rceil C_j \\ = R_j^W - C_j + C_j \\ = R_j^W. \end{aligned}$$

That is because of Eq. (3) and because task τ_j is schedulable under scheme W , so $R_j^W < D_j - J_j$; which means $R_j^W + J_j < D_j < T_j$. So, $\lceil (R_j^W + J_j)/T_j \rceil = 1$. $\square$

5. Example

The following simple example shows how $(D - J)$ -monotonic priority assignment improves the schedulability of a system. Suppose a system with two tasks as in Table 1.

It can be seen from this table that assigning priorities according to DM leads to τ_1 having higher priority than τ_2 and as a result τ_2 misses its first deadline. Fig. 1 illustrates this situation.

On the other hand, τ_1 and τ_2 will meet their deadlines if $(D - J)$ -monotonic priority assignment is employed; where τ_2 in this case will have higher priority than τ_1 . Fig. 2 illustrates how both tasks behave in the case of $(D - J)$ -monotonic priority assignment.

The worst case response time for τ_1 and τ_2 can be obtained by applying Eq. (1). Task τ_2 has the highest priority and hence its response time is equal to its worst case execution time (i.e., 3). For τ_1 , Eq. (1) is solved using a recurrence relationship starting with the value of its execution time (i.e., 6):

$$R_{\tau_1} = 6 + \left\lceil \frac{6 + 12}{25} \right\rceil 3 = 9,$$

Table 1
Tasks parameters

Task	C	J	D	T	$(D - J)$
τ_1	6	3	13	14	10
τ_2	3	12	20	25	8

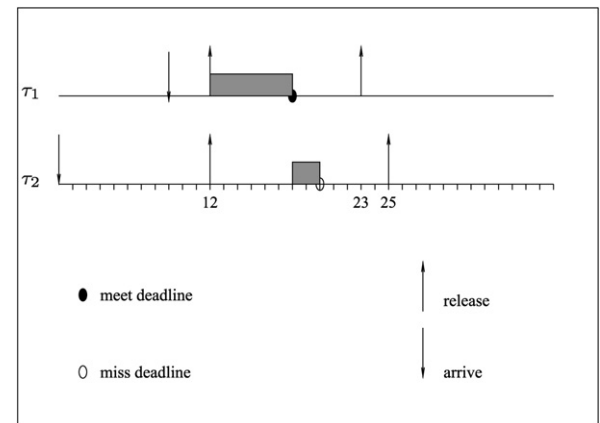


Fig. 1. Execution of τ_1 and τ_2 according to deadline monotonic priority assignment.

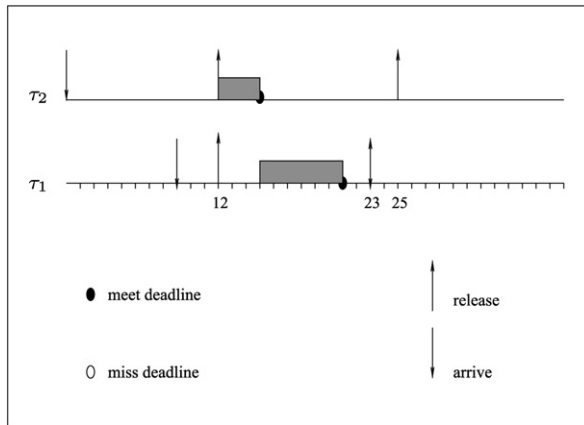


Fig. 2. Execution of τ_1 and τ_2 according to $(D - J)$ -monotonic priority assignment.

$$R_{\tau_1} = 6 + \left\lceil \frac{9 + 12}{25} \right\rceil 3 = 9.$$

So, $R_{\tau_1} = 9$. This is the worst case response time relative to its release. In terms of its response time relative to its arrival time, its own jitter value must be added (i.e., $9 + 3$). This value (i.e., 12) is less than the task's deadline (i.e., 13) which means that τ_1 is schedulable.

6. Conclusion

If tasks experience release jitter then DM priority assignment is no longer an optimal priority assignment

algorithm. It is shown in this paper that $(D - J)$ -monotonic is an optimal fixed priority policy when release jitter is present. The proof is given for tasks with deadlines less than or equal to their periods. The schedulability of a task set is improved when $(D - J)$ -monotonic priority assignment is employed when compared to other fixed priority policies.

References

- [1] J.W.S. Liu, Real Time Systems, Prentice-Hall, 2000.
- [2] C.L. Liu, J.W. Layland, Scheduling algorithm for multiprogramming in a hard real-time environment, Journal of the Association for Computing Machinery 20 (1) (1973) 46–61.
- [3] J.Y.T. Leung, J. Whitehead, On the complexity of fixed priority scheduling of periodic, real-time tasks, Performance Evaluation 2 (4) (1982) 237–250.
- [4] K. Tindell, Fixed priority scheduling of hard real-time systems, PhD thesis, University of York, 1993.
- [5] N.C. Audsley, A. Burns, On fixed priority scheduling, offsets and co-prime task periods, Information Processing Letters 67 (1998) 65–69.
- [6] N.C. Audsley, Optimal priority assignment and feasibility of static priority tasks with arbitrary start time, Tech. Rep. 164, University of York, 1991.
- [7] A. Burns, K. Tindell, A.J. Wellings, Effective analysis for engineering real-time fixed priority schedulers, IEEE Transactions on Software Engineering 21 (5) (1995) 475–480.
- [8] M. Joseph, P. Pandya, Finding response times in a real-time system, The Computer Journal 29 (5) (1986) 390–395.
- [9] N.C. Audsley, A. Burns, M. Richardson, K. Tindell, A.J. Wellings, Applying new scheduling theory to static priority preemptive scheduling, Software Engineering Journal 8 (5) (1993) 284–292.